

KEIKEU · DESKTOP RUNTIME · BEGINNER MANUAL

给新人的 Tauri + Vue 入门

不从脚手架学一套虚构项目。直接沿 keikeu 今天的 WebView、
Rust 宿主与 Python Sidecar，走完一次真实保存。

KEIKEU CONTRIBUTORS 简体中文 2026-08-11 VUE 3.5.40 TAURI 2.11 PDF 版

先记住这句话

keikeu 不是 “Vue 网站 + HTTP 后端”。Vue 只负责可见界面；Tauri/Rust 负责桌面生命周期、五个窄 command 与唯一 Sidecar；Python 才解释 Paper、Vault、Markdown 和 Index。

- 01 入口是 `index.html → main.js → App.vue`，当前导航只用一个 `ref`，没有 Router 或 Pinia。
- 02 项目自有的 Rust IPC 入口只有五个 command；`paper.save` 是装在 `bridge_request` 里的 Python protocol method。
- 03 写请求丢了响应时不能重发。App 保留 intent，重启 Sidecar 后只读对账。

事实边界：本手册解释当前实现，不定义新架构，也不授予修改 Vault、依赖、protocol 或发布配置的权限。运行时事实最终核对 PROJECT、SPEC、RULES、Architecture 与通过的测试。

目录

01	三个运行角色，一条本地链	02	启动项目时发生了什么
03	Vue 的最小语法	04	组件、状态与手写导航
05	Tauri/Rust 的五扇门	06	Sidecar 与 JSONL v2
07	走完一次 paper.save	08	失败、恢复与安全改动

三个运行角色，一条本地链

每层只做一类工作。新人判断文件归属时，先问“这是画界面、管桌面，还是处理作者文件？”

```

Vue 3 / WebView
  可见状态、表单、draft、导航
    | Tauri invoke
    ▼
Tauri / Rust core process
  窗口、原生能力、Sidecar 生命周期、单队列
    | stdin/stdout JSONL v2
    ▼
Python service + core
  DTO、业务编排、Vault、Markdown、Index
    |
    ▼
作者拥有的 Paper Markdown + 可重建 Index
  
```

Vue 是界面，不是文件服务

Vue 跑在操作系统 WebView 中。它知道当前显示 Paper、Library 还是 Vault，也知道输入框里的草稿；它不直接读取 Markdown，不拼 Vault 绝对路径，不写 Index JSON。Vue 收到的是 Python 已经整理好的 DTO。

Rust 是窄宿主，不是第二套业务后端

Rust 创建窗口，持有唯一 Python 子进程，按顺序收发 JSONL，并提供目录选择、Finder 打开和原生确认等桌面能力。它不解析卡页、不实现搜索、Trash 或迁移规则。

Python 才拥有产品与文件规则

`keikeu_bridge` 把 protocol 请求变成 application service 调用，并返回 DTO（只含跨层所需字段的数据对象）；`keikeu_core` 校验模型、路径和 Markdown。Markdown 是作者资产，Index 和设备状态只是可重建辅助数据。

不是 HTTP

开发时出现的 `http://127.0.0.1:1420` 只是 Vite 热更新页面。CSP 里的 `http://ipc.localhost` 是 Tauri IPC 所需来源。项目没有 localhost 业务服务器、REST API、WebSocket、账号或云后端。

启动项目时 发生了什么

完整桌面启动不是“开一个网页”。它要先准备 Python 可执行文件，再由 Tauri 同时组织 Vite、Rust 与 WebView。

第一次安装与启动

```
npm --prefix frontend ci
.venv/bin/python scripts/build_sidecar.py
npm --prefix frontend run tauri:dev
```

第一条按 lockfile 安装前端工具；第二条用 PyInstaller 生成带当前 Rust target triple 后缀的 Sidecar；第三条让 Tauri CLI 启动完整桌面链。Python 代码改变后，做 Tauri smoke 前要重新构建 Sidecar。

启动顺序

- 01 `tauri:dev` 读取 `frontend/src-tauri/tauri.conf.json`。
- 02 `beforeDevCommand` 运行 `npm run dev`，Vite 固定监听 `127.0.0.1:1420`。
- 03 Rust 从 `main.rs` 进入 `lib.rs`，注册插件和五个 command。
- 04 `BridgeHandle` 建立唯一 worker，并启动打包好的 `keikeu-sidecar`。
- 05 Rust 发 `system.hello`，核对 protocol v2、session、应用与 Core 版本。
- 06 WebView 载入 Vite 页面；`main.js` 把 `App.vue` 挂到 `#app`。
- 07 `App.vue` 每 200ms 查询 runtime；只有状态为 `ready` 才显示业务页面。
- 08 `PaperView` 调 `startup.load`，进入 Paper 或 Vault/migration gate。

文件地图

入口	当前责任
frontend/index.html	提供 <code>#app</code> 与 ES module 入口。

入口	当前责任
frontend/src/main.js	挂载 App, 载入全局 CSS。
frontend/src/App.vue	runtime gate、页面切换、durable intent。
frontend/src/bridge.js	所有 Vue→Tauri IPC helper。
frontend/src-tauri/src/	Rust 入口、commands 与 Sidecar worker。
src/keikeu_bridge/	Python DTO、service、protocol、stdin/stdout。
src/keikeu_core/	模型、Markdown、Vault 与 Index。

两个常见误判

`npm --prefix frontend run dev` 只证明 Vite 页面能启动; `npm --prefix frontend run build` 只生成前端静态资产。两者都不证明 Rust、Sidecar、Vault 或真实桌面流程可用。断开 Vault 的开发样张只能在 DEV 模式用 `?prototype=1` 打开。

CHAPTER 03

Vue 的最小语法

当前界面靠单文件组件、响应式值和父子事件就够了。先读懂这些，不要先给项目安装一套新状态框架。

单文件组件有三块

```
<script setup>
import { ref } from "vue";
const count = ref(0);
</script>

<template>
  <button type="button" @click="count += 1">{{ count }}</button>
</template>

<style scoped>
button { font: inherit; }
</style>
```

`<script setup>` 中的顶层 import、变量和函数可直接用于 template。`scoped` 让样式主要作用于当前组件；共享 token 与全局底色 仍在 `frontend/src/style.css`。

`ref`、`computed`、`watch`

`ref(initial)`

可变化的响应式盒子。脚本中用 `status.value`，模板中写 `status`。

`computed(() => ...)`

从其他状态推导只读结果。例如 active page、dirty、busy；getter 不做请求或写盘。

`watch(source, callback)`

在明确变化后同步状态或发事件。项目用它更新草稿 baseline、选择项和 dirty 通知。

模板中最常见的符号

写法	白话	项目例子
<code>{{ value }}</code>	把值显示成文字。	runtime 版本、Paper code、错误信息。
<code>v-if / v-else-if</code>	按状态选择一块界面。	App 的 runtime gate、Vault 的 picker/migration。
<code>v-for="item in items"</code>	为列表每项生成结构。	Paper 页码、Library 项目；同时给稳定 <code>:key</code> 。
<code>:disabled="busy"</code>	把 JS 值绑定到 HTML 属性。	请求期间禁止重复点击。
<code>@click="save"</code>	监听浏览器事件。	保存、切页、打开 Vault。
<code>v-model="draft.name"</code>	输入值与响应式状态双向同步。	名称、正文、Tags、select、checkbox。

props 向下，events 向上

```
// child
const props = defineProps({ paper: { type: Object, required: true } });
const emit = defineEmits(["save"]);
emit("save", draft);

<!-- parent -->
<PaperV4Workbench :paper="paper" @save="savePaper" />
```

props 是父组件给孩子的输入；孩子用 emit 告诉直接父组件发生了什么。组件事件不像 DOM 点击那样自动向祖先冒泡，所以跨层动作必须显式接力。

生命周期与真实 DOM

`onMounted` 在组件进入页面后注册 listener 或读取启动状态；`onUnmounted`/`onBeforeUnmount` 清 timer 和 listener；`onActivated` 处理 KeepAlive 缓存组件再次出现。模板上的 `ref="contentInput"` 配合同名 `ref(null)` 取得真实元素，`nextTick` 等 DOM 更新完再聚焦。

CHAPTER 04

组件、状态与手写导航

keikeu 只有三个生产页面，根组件的一个字符串已经能表达导航；Router 和全局 Store 现在只会增加新概念。

```
App.vue
├─ PaperView.vue
│   └─ PaperV4Workbench.vue
├─ LibraryView.vue      <KeepAlive>
│   └─ LibraryV4Projection.vue
└─ VaultView.vue
```

根组件拥有跨页面状态

`destination` 只取 `paper`、`library`、`vault`；template 用 `v-if` 显示对应页面。`status` 控制启动/ready/blocked gate；`paperPath` 与 startup DTO 负责页面交接；`pendingIntent` 保存可能已经写盘但结果未知的操作。

容器组件与展示组件

组件	可以做	不可以做
<code>PaperView</code>	加载、保存、删除、错误/修复状态。	直接写 Markdown。
<code>LibraryView</code>	查询、移动、Branch、Trash 与系统交接。	自己解析 Index 文件。
<code>VaultView</code>	目录意图、预览、迁移确认、重启读回。	把 picker 返回值当作已验证 Vault。
<code>PaperV4Workbench</code>	草稿、页切换、表单反馈、emit save。	import bridge 或持久化。

组件	可以做	不可以做
<code>LibraryV4Projection</code>	筛选和展示父组件给的 DTO。	读取作者文件。

为什么 Library 使用 KeepAlive

离开 Library 时保留它的选择和筛选状态，再回来由 `onActivated` 刷新。切换 Vault 或 Sidecar 重启后，App 增加 `libraryContextGeneration`，用新的 key 丢弃旧缓存，避免显示另一个 Vault 的上下文。

最危险的短路

App 会把 `appRequest` 作为 `request` prop 传给页面。durable 操作必须调用 `props.request`，不能在页面里直接调用 `bridgeRequest`；绕过根 wrapper 会让 `pendingIntent` 消失，写入丢响应后便无法安全对账。测试可注入 fake request，production 仍走同一入口。

CHAPTER 05

Tauri/Rust 的五扇门

Vue 不直接拿系统权限。本项目只显式注册五个自有 Tauri command；业务 method 是下一层协议，不是更多后门。

从 JavaScript 调 Rust

```
import { invoke } from "@tauri-apps/api/core";

const response = await invoke("bridge_request", {
  method: "startup.load",
  params: {},
});
```

`invoke` 返回 Promise。Tauri 默认把 JavaScript 的 camelCase 参数名映射到 Rust snake_case，因此 `relativeTarget` 对应 `relative_target`。返回类型经 Serde 序列化为前端可读数据。

五个项目自有 command

TAURI COMMAND	作用	是否进入 PYTHON PROTOCOL
<code>runtime_status</code>	读取 Rust 持有的 starting/ready/blocked/stopped。	否。

TAURI COMMAND	作用	是否进入 PYTHON PROTOCOL
<code>bridge_request</code>	承载批准的业务 method 与 params。	是。
<code>restart_sidecar</code>	杀死旧 child、建立新进程、新 session 与 hello。	只做内部 hello。
<code>choose_vault_directory</code>	打开原生文件夹 picker，返回选择意图。	否；Vue 随后必须调 <code>vault.inspect</code> 。
<code>open_system_target</code>	让 Python 验证相对目标，再由 opener 打开或在 Finder 显示。	内部调用 <code>system.resolve_target</code> 。

command 与 protocol method 不是一回事

`paper.save`、`library.query`、`vault.inspect` 都是 `bridge_request` payload 内的 method。Rust 的 `public_policy` 先拒绝未知 method 并决定 read/mutation timeout；Python dispatcher 再独立核对 method、params 与 session。`system.hello` 和 `system.resolve_target` 是宿主内部方法，不能借通用 bridge 从 Vue 直达。

配置、插件与权限

frontend/src-tauri/tauri.conf.json

关联 Vite dev URL、production dist、窗口尺寸、CSP、bundle 与 externalBin。

frontend/src-tauri/src/lib.rs

初始化 shell/dialog/opener，manage BridgeHandle，并在同一个 `generate_handler!` 注册五个 command。

frontend/src-tauri/capabilities/main.json

只给 main 窗口当前需要的 event、destroy 与 dialog 权限；不是业务 method allowlist 的替代品。

frontend/src-tauri/Cargo.toml

固定 Rust edition、最低 Rust 与精确 Tauri/Serde/plugin 依赖。

新增插件不是一行 IMPORT

一个 Tauri 插件通常同时牵涉 npm 包、Cargo crate、Rust 初始化、capability、CSP、平台与打包。当前任务没有第二个真实需求时，不加插件、不造 abstraction。

Sidecar 与 JSONL v2

Sidecar 是 Tauri 随 app 携带、由 Rust 启停的 Python 可执行文件；双方用标准输入输出交换一行一请求的 JSON，而不是开端口。

它怎样被打包和找到

scripts/build_sidecar.py 读取 `rustc -Vv` 的 host triple，用 PyInstaller `--onefile` 生成类似 `keikeu-sidecar-aarch64-apple-darwin` 的文件。`tauri.conf.json` 的 `externalBin` 只写基名 `binaries/keikeu-sidecar`；Tauri 按目标架构寻找带后缀的产物。

一次握手

```
{"v":2,"id":1,"method":"system.hello","params":{}}
```

```
{
  "v":2,"id":1,"ok":true,"result":{
    "protocol_version":2,
    "session_id":"opaque-session",
    "app_version":"0.1.0",
    "core_version":"paper-v4/index-v4"
  }}

```

hello 没有 session。Python 每次 hello 清空旧 preview/edit/migration handle，再签发新 session；后续每个请求都必须携带它。旧 token 因此不会跨进程重启继续生效。

为什么只有一个 worker

多个 Tauri command 可以同时到来，但 `BridgeHandle` 把 `WorkerJob` 放进同一个 Rust channel。worker 一次只写一个 JSONL request，并等到匹配 id 的 response 后才处理下一个。这个简单顺序避免响应串线，也让 mutation 只执行一次。

Python 的四层

01 `sidecar.py` 只创建 service，并把 stdin/stdout 交给 `run_jsonl`。

02 `protocol.py` 校验唯一 JSON key、版本、id、session、method 与 params。

03 `service.py` 把 DTO 编排成 startup、Vault、Paper、Library 或 migration workflow。

STDOUT 是协议线

Sidecar stdout 每个输入行必须恰好返回一个 JSON response 并 flush。在 Python 路径里随手 `print()` 会让 Rust 把日志当协议响应，继而阻塞 runtime。即使用 stderr 诊断，也绝不能记录作者正文、Vault 私密路径或 token。

CHAPTER 07

走完一次 `paper.save`

“点保存”不是一个函数。它是一条逐层收窄权限的链：组件保草稿，根组件保恢复意图，Rust 保请求顺序，Python 保作者文件。

先把两个词翻成白话：`baseline` 是最后一次可信的已保存快照；`intent` 是“刚才想写什么”以及事后只读核对所需的数据，不是重试指令。

- 01 `PaperV4Workbench.save()` 做即时表单检查，并 emit `save`；它仍然不碰 bridge。
- 02 `PaperView.savePaper()` 从 Paper DTO 组装 `edit_token`、`vault_locator`、名称、Tags 与有序 pages。
- 03 `PaperView` 同时保存旧 baseline、submitted draft、target path 与 source digest，形成可只读对账的 intent。
- 04 页面调用父组件注入的 `props.request("paper.save", save, intent)`。
- 05 `App.appRequest()` 先把 durable intent 放进根状态，再调用 `bridgeRequest`。
- 06 `bridge.js` invoke 唯一 Tauri command `bridge_request`，Python 错误 envelope 会被转换为结构化 throw。
- 07 Rust policy 将 `paper.save` 分类为 mutation，附加 session 和递增 id，经单 worker 写入一行 JSON。
- 08 Python dispatcher 严格检查字段，构造 `PaperSaveDto`，只调用 service 一次。
- 09 service 校验当前 Vault locator 与 edit token，把 DTO 转成 Paper 模型。
- 10 新 Paper 用无覆盖创建；既有 Paper 用 source bytes 做 compare-and-swap，外部修改会拒绝覆盖。

- 11

写后重新读取验证；Markdown 已成功而 Index 重建失败时，返回成功加 `index_degraded` warning。
- 12

响应沿 JSONL→Rust→Tauri IPC→Vue 返回；Vue 用返回 Paper 建立新 baseline，确认成功后 App 才清 intent。

为什么要同时传 token、locator 与 digest

值	防止什么
<code>edit_token</code>	把保存绑定到 Python 已经打开或创建的具体编辑快照。
<code>vault_locator</code>	Sidecar 重启或 Vault 切换后，不对错误 Vault 读取或写入。
<code>source_digest / source bytes</code>	外部编辑器改过文件时拒绝静默覆盖。
<code>submitted + baseline</code>	未知结果后区分 committed、not_committed、stale 与 repair_required。

INDEX 不是回滚理由

Paper Markdown 已安全替换后，Index 更新失败不能把保存说成失败，也不能重发 `paper.save`。Vue 接受新的 Paper baseline，显示 `index_degraded`，只允许用户显式重建可丢弃的 Index。

CHAPTER 08

失败、恢复与安全改动

简单架构不等于忽略失败。写入是否发生过必须分清；不确定时宁可停住，也不能再按一次。

四个容易混淆的状态

状态	已知事实	下一步
<code>stale_snapshot</code>	保存前已证明磁盘内容变了，目标写入未执行。	保留草稿，重新打开并人工决定。

状态	已知事实	下一步
<code>commit_unknown</code>	mutation 发出后响应丢失，磁盘可能改过，也可能没改。	阻塞写入、重启、只读对账；禁止 replay。
<code>repair_required</code>	磁盘 Paper 结构损坏，不能安全作为可编辑 DTO 打开。	显示路径和结构原因，人工修复后重新检查。
<code>index_degraded</code>	作者 Markdown mutation 已知成功，辅助 Index 不可靠。	接受新 baseline，需要时显式重建 Index。

`commit_unknown` 的恢复路线

```
paper.save 响应丢失
→ Rust 杀死 Sidecar并把 runtime 设为 blocked
→ App 根状态保留 pendingIntent
→ 用户显式“重启本地 Core”
→ 新 hello / 新 session
→ PaperView 调 paper.reconcile_save (只读)
→ 根据磁盘返回 committed / not_committed / stale / repair_required
→ 只有 committed / not_committed 才清 intent
```

四层验证各管一段

层	当前验证	不能冒充
Vue	可见必填、按钮 busy、DTO 基本形状、离开保护。	最终领域/路径权威。
Rust	五个 command、method allowlist、timeout、id/session/response envelope。	Paper 或 Markdown 规则。
Python protocol/service	严格 JSON 字段、DTO、locator、workflow 与稳定错误。	GUI 状态。
Python core	模型、Home containment、symlink、CAS、原子文件操作。	桌面窗口生命周期。

第一次改动怎么选检查

```
# Vue 组件与 bridge mock
npm --prefix frontend run test
npm --prefix frontend run build

# Rust command / worker
cargo test --manifest-path frontend/src-tauri/Cargo.toml

# Python protocol / service / core
.venv/bin/python -m pytest
.venv/bin/python -m compileall -q src

# 完整桌面链前重建 Sidecar
.venv/bin/python scripts/build_sidecar.py
npm --prefix frontend run tauri:dev
```

Vue 测试在 jsdom 中 mock Tauri IPC，适合证明组件和 envelope 行为；Vite build 证明前端可构建；Cargo test 证明 Rust worker 与失败分类；pytest 证明 Python 契约和文件逻辑。它们都不能单独冒充最终 Tauri 启动、真实平台流程或产品验收。

新人第一练习

1. 从 frontend/src/main.js 跟到 `App.vue`，解释为什么 starting 时看不到 PaperView。
2. 从 `PaperV4Workbench` 的保存按钮跟到 Python `paper_save`，每层只写一句责任。
3. 只修改一处不改变行为的 UI 文案，跑 focused Vitest 与 Vite build，检查 diff 后撤销练习。

现在明确跳过

不加 Router、Pinia、TypeScript、HTTP、Axios、WebSocket、UI kit、文件 watcher 或第二条 Sidecar 通道。等第二个真实需求证明现有 `ref + props + events` 或单队列不够，再做一次有证据的架构决定。

来源与继续阅读

本页于 2026-08-11 依据当前仓库源码审计。官方链接解释框架通用行为；它们不能覆盖 keikeu 自己的职责边界。

项目事实

1. frontend/src/main.js、App.vue、bridge.js
2. PaperView.vue、PaperV4Workbench.vue、LibraryView.vue、VaultView.vue
3. frontend/src-tauri/src/lib.rs、commands.rs、bridge.rs、tauri.conf.json、capabilities/main.json
4. src/keikeu_bridge/protocol.py、service.py、sidecar.py
5. SPEC、RULES、PROJECT、ADR-0004

Vue 官方文档

1. Quick Start 与 `<script setup>`
2. Reactivity Fundamentals、Computed、Watchers
3. Props、Component Events、Lifecycle Hooks、KeepAlive

Tauri 官方文档

1. Process Model 与 Project Structure
2. Calling Rust from the Frontend 与 Embedding External Binaries
3. Vite frontend、Capabilities、Content Security Policy

最后核对：manual 是理解材料。产品范围看 SPEC；工程与数据纪律看 RULES；当前阶段、版本与已运行证据看 PROJECT。源码改变后，本页必须跟着校准。